

UNIT 3: C PROGRAMMING

01 - 1D Arrays & Array Operations

 **Aim:** To understand what an array is, visualize contiguous memory allocation and zero-based indexing, read array elements dynamically from user input at runtime, display them, and perform standard array operations.

1. WHAT IS AN ARRAY? & MEMORY ALLOCATION

An **array** is a collection of multiple variables of the **same data type** stored sequentially in **contiguous (side-by-side) memory locations** under a single common name.

Syntax: `data_type array_name[size];` → e.g., `int a[5];` reserves 5 continuous integer blocks ($5 \times 4 \text{ bytes} = 20 \text{ bytes}$ in RAM).

👁 Memory Architecture & Index Mapping for `int a[5]`:

12	25	38	41	59
<code>a[0] (Base)</code>	<code>a[1]</code>	<code>a[2]</code>	<code>a[3]</code>	<code>a[4] (Last)</code>
#2000	#2004	#2008	#2012	#2016

• **Index starts at 0:** First index is 0, last index is `size - 1` (here: $5 - 1 = 4$).

• **Address Offset:** Jumps by **+4 bytes** per cell because `sizeof(int) = 4 bytes`.

2. RUNTIME INPUT & DISPLAY PROGRAM (STEP-BY-STEP)

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter number of elements:
");
    scanf("%d", &n);

    int a[n];
    // 1. Reading values at Runtime
    printf("Enter %d numbers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]); // & passes
        memory address
    }

    // 2. Printing elements along with
    their indices
    printf("\nStored Array
Elements:\n");
    for (int i = 0; i < n; i++) {
        printf("Element at index [%d] =
%d\n", i, a[i]);
    }
}
```

💻 Terminal Execution (Runtime):

```
Enter number of elements: 5
Enter 5 numbers:
12 25 38 41 59
```

Stored Array Elements:

```
Element at index [0] = 12
Element at index [1] = 25
Element at index [2] = 38
Element at index [3] = 41
Element at index [4] = 59
```

🔍 How Runtime Input Works

- When `i = 0`, `scanf("%d", &a[0])` stores 12 directly in address #2000.
- When `i = 1`, `scanf("%d", &a[1])` stores 25 in address #2004, and so on.

```
return 0;  
}
```

3. COMMON ARRAY OPERATIONS

Operation	What it does?	How to write the loop code?
1. Traversal	Visiting each element once (e.g., to print or count).	<pre>for(int i = 0; i < n; i++) { printf("%d ", a[i]); }</pre>
2. Sum & Average	Accumulating all values to compute totals.	<pre>sum += a[i]; → float avg = (float)sum / n;</pre>
3. Search (Linear)	Checking if a specific target value exists in the array.	<pre>if (a[i] == key) { printf("Found at index %d", i); break; }</pre>
4. Maximum / Minimum	Comparing each element to find highest or lowest.	<pre>if (a[i] > max) { max = a[i]; } (Initialize max = a[0])</pre>

⚠ **Out of Bounds Danger (No Bounds Checking):**
In C, if size is 5, accessing `a[5]` or `a[10]` is illegal! C won't show an error at compile time, but it reads/overwrites random RAM data (producing **garbage values** or program crashes)!

💡 **Why Arrays Start at 0:**
Index `i` actually acts as an **offset formula**:
 $\text{Address} = \text{Base_Address} + (\text{Index} \times \text{Size_of_Data})$
For index 0: $\#2000 + (0 \times 4) = \#2000$ (Exact start point with zero shift!).



One-Line Summary: 1D Arrays store multiple same-type items side-by-side in RAM; indices run from 0 to `n-1`, accessed using `a[i]`.

UNIT 3: C PROGRAMMING

PAGE 2: 2D ARRAYS (MATRICES) – INPUT, LOGIC & OPERATIONS

 **Aim:** To master Two-Dimensional (2D) Arrays, understand tabular row-column addressing, read and display matrices dynamically at runtime using nested loops, and perform common matrix operations.

1. WHAT IS A 2D ARRAY? & TABULAR LAYOUT

A **2D Array** is an array of arrays organized in a grid of **Rows** and **Columns** (commonly called a **Matrix**).

Syntax: `data_type array_name[rows][cols];` → e.g., `int a[2][3];` reserves $2 \times 3 = 6$ integer elements ($6 \times 4 \text{ bytes} = 24 \text{ bytes}$).

👁 **Logical Grid Organization for `int a[2][3];`**

	Col 0	Col 1	Col 2
Row 0	10 <code>a[0][0]</code>	20 <code>a[0][1]</code>	30 <code>a[0][2]</code>
Row 1	40 <code>a[1][0]</code>	50 <code>a[1][1]</code>	60 <code>a[1][2]</code>

- **Index Formula:** `a[row_index][col_index]`
- **Row Index:** Runs from 0 to rows - 1
- **Col Index:** Runs from 0 to cols - 1
- **RAM Storage:** Stored in contiguous **Row-Major Order** (Row 0 elements stored first, followed immediately by Row 1).

2. RUNTIME INPUT & MATRIX DISPLAY (COMPLETE PROGRAM)

```
#include <stdio.h>
int main() {
    int a[2][2], r, c;
    printf("Enter matrix elements (2x2):\n");
    // 1. Reading runtime values using Nested Loops
    for (r = 0; r < 2; r++) { // Outer loop: Rows
        for (c = 0; c < 2; c++) { // Inner loop: Columns
            scanf("%d", &a[r][c]);
        }
    }
    // 2. Printing in proper 2D Matrix Format
    printf("\nThe 2x2 Matrix is:\n");
    for (r = 0; r < 2; r++) {
        for (c = 0; c < 2; c++) {
```

Terminal Execution (Runtime):

Enter matrix elements (2x2):

1 2
3 4

The 2x2 Matrix is:

1 2
3 4

Why Nested Loops?

- **Outer Loop (r):** Selects the horizontal row.
- **Inner Loop (c):** Sweeps left-to-right through every column in that row.
- **`printf("\n");`:** Placed right after the inner loop finishes to start the next row on a fresh line!

```

printf("%d\t", a[r][c]); // Tab
space
}
printf("\n"); // Line break after
each row
}
return 0;
}

```

3. COMMON 2D ARRAY OPERATIONS

Operation	Key Purpose	Standard Loop Logic
1. Matrix Addition	Adds matching elements from two identical-sized matrices ($C = A + B$)	<code>sum[i][j] = a[i][j] + b[i][j];</code>
2. Matrix Transpose	Swaps rows into columns ($B[j][i] = A[i][j]$)	<code>transpose[j][i] = a[i][j];</code>
3. Diagonal Sum (Trace)	Sums main diagonal elements where row index equals column index ($i == j$)	<code>if (i == j) { diag_sum += a[i][j]; }</code>
4. Row-wise Sum	Computes total sum of each individual row independently	Reset <code>row_sum = 0;</code> before inner loop, print after.

⚠ Index Out-of-Bounds Trap:

In `int a[2][3]`, the valid row indices are only 0, 1 and column indices are 0, 1, 2. Writing `a[2][3] = 10;` causes memory corruption!

💡 Matrix Initialization Trick:

You can initialize directly: `int a[2][2] = {{1, 2}, {3, 4}};`.
In C, you can omit the row dimension, but you **must** provide column dimension: `int a[][2] = {1, 2, 3, 4};`.



One-Line Summary: A 2D array represents data in rows and columns; nested loops (outer for rows, inner for columns) are used to input, display, and process elements.

UNIT 3: C PROGRAMMING

PAGE 3: STRINGS – CONCEPT, MEMORY LAYOUT & RUNTIME INPUT

 **Aim:** To understand how text is stored in C as a 1D character array terminated by a Null character ('\\0'), accept string inputs at runtime, and compare `scanf()` with `fgets()`.

1. WHAT IS A STRING IN C? & MEMORY LAYOUT

In C, there is no separate `string` keyword like in Java or Python. A string is simply a **1D array of characters** that ends with a special sentinel value called the **Null Terminator ('\\0')**. The null character has an ASCII value of 0 and signals the end of the text to functions like `printf()`.

👁 **Memory Representation of `char name[6] = "INDIA";` (Each `char` = 1 byte):**

'I'	'N'	'D'	'I'	'A'	'\\0'
name[0]	name[1]	name[2]	name[3]	name[4]	name[5]
#3000	#3001	#3002	#3003	#3004	#3005

👉 **Rule of Thumb:** An N-character word always needs an array size of at least (N + 1) to store the terminating '\\0'!

2. RUNTIME INPUT & DISPLAY PROGRAM

```
#include <stdio.h>

int main() {
    char firstName[30];
    char fullName[50];

    // 1. Reading single word (stops
    at space)
    printf("Enter first name: ");
    scanf("%s", firstName); // No &
    needed!

    // Clear newline buffer before
    reading multi-word string
    getchar();

    // 2. Reading complete line with
    spaces safely
    printf("Enter full name: ");
    fgets(fullName,
    sizeof(fullName), stdin);

    // 3. Printing strings
```

 **Terminal Execution (Runtime):**
Enter first name: **Mukilan**
Enter full name: **Mukilan Selvaraj**

--- Profile ---
First Name: Mukilan
Full Name : Mukilan Selvaraj

Why No & in `scanf("%s", str)`?

In C, the array name by itself (`firstName`) already represents the **base memory address** of the array (`&firstName[0]`). Adding an `&` is redundant.

```

printf( "\n--- Profile ---\n");
printf("First Name: %s\n",
firstName);
printf("Full Name : %s",
fullName);

return 0;
}

```

3. INPUT METHODS COMPARISON

Input Function	Handles Spaces?	Buffer Overflow Safety	Best Use Case
scanf("%s", str);	❌ No (stops reading at space/tab/newline)	Unsafe (can overflow if input is too long)	Single words (names, codes, IDs)
fgets(str, n, stdin);	✅ Yes (reads until enter key is pressed)	✅ Safe (reads at most n - 1 characters)	Full sentences, addresses, full names
gets(str);	✅ Yes	❌ Removed from modern C (dangerous!)	Never use in modern programming

⚠️ Forgetting the Null Character:

If you assign characters manually: `char s[3] = {'H', 'i'};` without adding `'\0'`, `printf("%s", s)` keeps reading past the array until it randomly finds a 0 in memory, printing strange junk characters!

💡 Single Quotes vs Double Quotes:

- `'A'` → Single character constant (1 byte).
- `"A"` → String literal containing two bytes: `'A'` and `'\0'!`

 **One-Line Summary:** Strings are 1D character arrays terminated by `'\0'`; use `scanf("%s")` for single words and `fgets()` for lines containing spaces.

UNIT 3: C PROGRAMMING

PAGE 4: STANDARD STRING FUNCTIONS (<string.h>)

 **Aim:** To master standard C library string manipulation functions — **strlen()**, **strcpy()**, **strcat()**, and **strcmp()** — and observe their operations at runtime.

1. THE 4 ESSENTIAL STRING FUNCTIONS IN C

Since C cannot directly assign or concatenate strings using operators like = or +, the standard header **<string.h>** provides pre-built functions to perform these common operations.

1. strlen(str) – String Length

Counts all characters until it hits '\0'.

Note: The null terminator is **never** counted!

2. strcpy(dest, src) – String Copy

Copies text from src into dest.

Rule: dest must have enough space!

3. strcat(dest, src) – Concatenate

Attaches src to the end of dest.

Overwrites dest's old '\0' with src.

4. strcmp(str1, str2) – Compare

Compares ASCII character by character.

Returns 0 if both strings match identically!

2. COMPREHENSIVE PRACTICAL PROGRAM (RUNTIME DEMONSTRATION)

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50], str2[50],
    copyStr[50];

    printf("Enter first string: ");
    scanf("%s", str1);
    printf("Enter second string: ");
    scanf("%s", str2);

    // 1. Length of string
    printf("\nLength of str1 = %lu\n", strlen(str1));

    // 2. Copying string
    strcpy(copyStr, str1);
    printf("Copied string = %s\n", copyStr);

    // 3. String Comparison
    if (strcmp(str1, str2) == 0) {
```

Terminal Execution (Runtime):

Enter first string: **Hello**
Enter second string: **World**

Results:

Length of str1 = 5
Copied string = Hello
Comparison = Strings are DIFFERENT
Concatenated = HelloWorld

How strcmp() Evaluates Values

- **strcmp(s1, s2) == 0** → Exact match.
- **strcmp(s1, s2) > 0** → First differing character in s1 has larger ASCII value.
- **strcmp(s1, s2) < 0** → First differing character in s1 has smaller ASCII value.

```

        printf("Comparison = Strings
are EQUAL\n");
    } else {
        printf("Comparison = Strings
are DIFFERENT\n");
    }

    // 4. Concatenation (joining)
    strcat(str1, str2);
    printf("Concatenated = %s\n",
str1);

    return 0;
}

```

3. QUICK REFERENCE MATRIX FOR <STRING.H>

Function Prototype	Destination Buffer Requirement	Return Value	Common Mistake to Avoid
size_t strlen(const char *s)	Read-only (doesn't modify)	Number of characters before '\0'	Confusing strlen() with sizeof()
char *strcpy(char *dest, char *src)	Must hold at least strlen(src) + 1	Pointer to dest	Reversing args: strcpy(src, dest) ❌
char *strcat(char *dest, char *src)	Must hold strlen(dest) + strlen(src) + 1	Pointer to dest	Target array too small (Buffer Overflow)
int strcmp(char *s1, char *s2)	Read-only (neither is modified)	0, positive integer, or negative integer	Using s1 == s2 (which only compares RAM addresses!)

⚠ The "s1 == s2" Exam Trap:

Writing `if (str1 == str2)` in C checks if both strings live at the **exact same memory address**, NOT if their text contents match! Always use `strcmp(str1, str2) == 0` to test textual equality.

💡 strlen() vs sizeof():

For `char s[10] = "Hi";`:

- `strlen(s)` returns 2 (counts letters up to '\0').
- `sizeof(s)` returns 10 (total allocated bytes in RAM).

 **One-Line Summary:** Include `<string.h>` to use `strlen` (length), `strcpy` (copy), `strcat` (append), and `strcmp` (compare content).

UNIT 3: C PROGRAMMING

PAGE 5: POINTERS – ADDRESS-OF (&) AND DEREFERENCING (*)

 **Aim:** To demystify pointers, understand memory addresses, master pointer declaration and initialization, and read/modify variable values directly through dereferencing.

1. WHAT IS A POINTER? & PHYSICAL MEMORY VIEW

A **pointer** is a special variable that **stores the memory address** of another variable instead of a regular data value.

Syntax: `data_type *pointer_name;` → e.g., `int *ptr = #` tells the compiler that `ptr` holds the address of integer `num`.

 **Pointer Variable (ptr)**

#2004

Lives at Address: #5000

points to (&num)



*ptr fetches value

 **Target Variable (num)**

25

Lives at Address: #2004

1. Address-Of Operator (&)

Extracts the unique RAM address where a variable is located.
`&num` → Gives #2004 (Printed with format specifier %p).

2. Dereference Operator (*)

Goes to the address stored in the pointer and extracts or modifies the data inside.
`*ptr` → Reads or overwrites value 25.

2. RUNTIME INPUT & POINTER MANIPULATION PROGRAM

```
#include <stdio.h>

int main() {
    int num;
    int *ptr = &num; // ptr stores
the address of num

    printf("Enter a number: ");
    scanf("%d", &num); // Reading
runtime input

    printf("\n--- Inspecting Values
---\n");
    printf("Direct Value (num) =
%d\n", num);
    printf("Address of num (&num) =
%p\n", &num);
    printf("Stored in ptr (ptr) =
%p\n", ptr);
    printf("Value via pointer (*ptr)
= %d\n", *ptr);
```

Terminal Execution (Runtime):

Enter a number: 25

--- Inspecting Values ---

Direct Value (num) = 25
Address of num (&num) = 0x7ffd2004
Stored in ptr (ptr) = 0x7ffd2004
Value via pointer (*ptr) = 25

After `*ptr += 10`, num = 35

What Just Happened?

- `ptr` holds 0x7ffd2004 (the RAM address of `num`).
- Writing `*ptr = 35` opens box 0x7ffd2004 and changes its contents to 35.
- `num` now evaluates to 35 because both refer to the exact same memory cell!

```

// Modifying original variable
through pointer
*ptr = *ptr + 10; // num is
updated directly in RAM
printf("\nAfter *ptr += 10, num
= %d\n", num);

return 0;
}

```

3. COMMON POINTER SYNTAX QUICK REFERENCE

Expression	Meaning / Action	Type of Output
<code>int *p;</code>	Declaration — creates a pointer variable designed to point to an <code>int</code> .	Pointer definition
<code>p = &x;</code>	Assigns the address of <code>x</code> to pointer <code>p</code> .	Address assignment
<code>p</code>	The memory address stored inside <code>p</code> .	Memory address (hex)
<code>*p</code>	Dereferencing: Accesses or modifies the value at address <code>p</code> .	Integer value
<code>&p</code>	The memory address where the pointer variable itself is located in RAM.	Pointer's own address

⚠ Wild / Uninitialized Pointer Danger:

Declaring `int *p;` without setting an address makes it point to a random memory location! Writing `*p = 50;` can crash your operating system. Always initialize with `NULL`: `int *p = NULL;`!

💡 Two Uses of the Asterisk (*):

- **In Declaration:** `int *p;` → declares a pointer type.
- **In Executable Code:** `*p = 20;` → dereferences (fetches/updates value).



One-Line Summary: `&` gives the memory address of a variable; a pointer stores that address; `*` accesses or updates the value at that address.

UNIT 3: C PROGRAMMING

PAGE 6: POINTER ARITHMETIC & ARRAY TRAVERSAL

 **Aim:** To perform arithmetic operations on single values using dereferenced pointers ($*p1 + *p2$), understand pointer scaling rules, and traverse 1D arrays using pointer offsets $*(ptr + i)$.

1. ARITHMETIC OPERATIONS ON SINGLE VALUES VIA POINTERS

By applying dereferencing ($*$), pointers allow us to perform standard arithmetic (addition, subtraction, multiplication) directly on the values stored in separate memory locations:

```
#include <stdio.h>

int main() {
    int a, b, sum, diff, prod;
    int *p1 = &a, *p2 = &b;

    printf("Enter two numbers: ");
    scanf("%d %d", p1, p2); //
    p1=&a, p2=&b

    sum = *p1 + *p2; //
    Dereferencing to add values
    diff = *p1 - *p2; //
    Dereferencing to subtract
    prod = (*p1) * (*p2);

    printf("Sum = %d\n", sum);
    printf("Difference = %d\n",
diff);
    printf("Product = %d\n", prod);
    return 0;
}
```

Terminal Output (Arithmetic on Values):

```
Enter two numbers: 20 8
Sum = 28
Difference = 12
Product = 160
```

Dereferencing vs Address Operations

- $*p1 + *p2$: Extracts stored numbers ($20 + 8 = 28$).
- $p1 + p2$: Attempting to add addresses directly is **illegal** in C!

2. POINTER SCALING RULE & ADDRESS JUMPS

When you add an integer to a pointer ($ptr + 1$), C scales the shift by the size of the target data type:

New Address = Current Address + (n × sizeof(data_type))

👁 Visualizing `int *ptr = arr;` → Each step jumps by +4 bytes:

10

`*(ptr) = arr[0]`

#2000

20

`*(ptr + 1) = arr[1]`

#2004

30

*(ptr + 2) = arr[2]

#2008

40

*(ptr + 3) = arr[3]

#2012

If ptr = #2000, then ptr + 1 = #2004 (not #2001), because sizeof(int) == 4.

3. RUNTIME INPUT & ARRAY TRAVERSAL USING POINTERS

```
#include <stdio.h>

int main() {
    int arr[4], n = 4;
    int *ptr = arr; // Points to arr[0]

    printf("Enter 4 integers:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", (ptr + i)); // No & needed!
    }

    printf("\nValues and Addresses in RAM:\n");
    for (int i = 0; i < n; i++) {
        printf("At address %p: Value = %d\n",
               (void*)(ptr + i), *
               (ptr + i));
    }
    return 0;
}
```

Terminal Output (Array Traversal):

Enter 4 integers:

10 20 30 40

Values and Addresses in RAM:

At address 0x7ffd2000: Value = 10

At address 0x7ffd2004: Value = 20

At address 0x7ffd2008: Value = 30

At address 0x7ffd200c: Value = 40

Array-Pointer Equivalence

In C, bracket notation is just pointer arithmetic:

- **arr[i]** is evaluated as ***(arr + i)**
- **&arr[i]** is identical to **(arr + i)**

4. PERMITTED VS. ILLEGAL POINTER OPERATIONS

Operation	Valid / Invalid?	Formula / Example Behavior
Increment (ptr++)	✓ Valid	Advances pointer forward by sizeof(type) bytes.
Decrement (ptr--)	✓ Valid	Steps pointer backward by sizeof(type) bytes.
Adding Integer (ptr + n)	✓ Valid	Moves pointer forward by n * sizeof(type) bytes.
Subtracting Pointers (p2 - p1)	✓ Valid	Returns the number of elements between two pointers.
Adding Two Pointers (p1 + p2)	✗ ILLEGAL	Adding two RAM addresses has no physical meaning!
Multiplication / Division	✗ ILLEGAL	Operations like ptr * 2 or ptr / 3 cause compiler errors.

⚠ Parentheses Precedence Trap:

- ***ptr++** → Reads value first, then increments pointer address.
- **(*ptr)++** → Increments the actual integer data value at that address!

💡 Constant Array Pointer:

An array name is a **constant pointer** to its base index. You can run ptr++, but you **cannot** run arr++ because arr cannot change its base memory location!

 One-Line Summary: Dereference pointers (`*p1 + *p2`) to compute arithmetic on single values; pointer math (`ptr + i`) scales by data type size to traverse contiguous arrays.

UNIT 3: C PROGRAMMING

PAGE 7: PREREQUISITE TO FUNCTION POINTERS – PARAMETER PASSING MECHANICS

🎯 **Foundational Principle:** Before studying Function Pointers, students must master parameter passing. In C, *strictly speaking, there is only Call by Value*. What is conventionally termed "Call by Reference" is practically implemented by passing variable addresses using pointers.

1. SIDE-BY-SIDE SWAPPING EXPERIMENT

1. Call by Value (Passes Duplicate Copy)

Actual parameters are copied into separate formal parameter slots. Originals in caller scope remain unchanged.

```
#include <stdio.h>

void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Before\n");
    printf("x = %d, y = %d\n", x, y);
    swap(x, y);
    printf("After\n");
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Before swapping:
x = 10, y = 20
After swapping:
x = 10, y = 20 (Values remained
unchanged!)
```

Why did swapping not happen?

When calling `swap(x, y)`, values are cloned across functions:

`main(): x = 10` _____

2. Call by Reference (Passes Memory Address)

Passing `&x` and `&y` allows the function to reach directly into the caller's memory via dereferencing.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Before\n");
    printf("x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("After\n");
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Before swapping:
x = 10, y = 20
After swapping:
x = 20, y = 10 (Swapping
succeeded!)
```

How does it work step-by-step?

`&x` and `&y` pass RAM addresses into pointers `a` and `b`.

```

→ swap(): a = 10
main(): y = 20
→ swap(): b = 20
Inside swap(): a = 20, b = 10.
Only the copies exchange. The original
variables remain x = 10, y = 20!

```

- `*a` means: Go to address in `a` → accesses `x`.
- **Step 1:** `temp = *a;` → `temp = 10`
- **Step 2:** `*a = *b;` → `x = 20` (updates original `x`)
- **Step 3:** `*b = temp;` → `y = 10` (updates original `y`)

2. COMPREHENSIVE COMPARISON TABLE & MEMORY RULES

Feature	Call by Value	Call by Reference (Using Pointers)
What is passed?	Value (duplicate data copy)	Address of the variable in RAM
Uses pointer?	No	Yes (<code>int *a, int *b</code>)
Original variable modified?	No (caller variables remain untouched)	Yes (direct mutation in memory)
Function parameter type	Normal variable: <code>int a</code>	Pointer variable: <code>int *a</code>
Function call syntax	<code>swap(x, y);</code>	<code>swap(&x, &y);</code>
Accessing value inside body	Direct evaluation: <code>a</code>	Dereference operator: <code>*a</code>
Easy-to-Remember Mantra	Value → Copy → No change in original	Address → Pointer → Original can be changed

3. IMPORTANT SYMBOLS & TRANSITION TO FUNCTION POINTERS

Expression	Exact Conceptual Meaning
<code>x</code>	Direct value stored in variable <code>x</code> (e.g., <code>10</code>)
<code>&x</code>	Unique memory address where variable <code>x</code> resides
<code>int *p</code>	Declares <code>p</code> as a pointer variable pointing to an integer
<code>*p</code>	Dereferencing: Fetches value at address held in <code>p</code>

Memory Diagram for `int *p = &x;`

Address: #2004

`x` → [10]

↑

`p` holds: #2004

`p` = address of `x` • `*p = 10` • `&x` = address of `x`

🔑 Connection to Function Pointers (Avoid this Confusion):

- **Pointer to a Variable (`int *p;`):** Stores the address of a *data variable* in RAM.

- **Pointer to a Function (`int (*fp)(int, int);`):** Stores the address of an *executable function* in the Code Segment.

Example: `int (*fp)(int, int) = add;` allows running `printf("%d", fp(10, 20));` to output 30!

4. THE FULL LEARNING PATHWAY

1. Functions → 2. Arguments & Params → 3. Call by Value → 4. Pointers → 5. Call by Reference → 6. Function Pointers → 7. Callbacks



One-Line Summary for Students: Call by Value passes a copy of the value, whereas Call by Reference passes the address of the variable using pointers, allowing the function to modify the original variable.

UNIT 3: C PROGRAMMING

PAGE 8: DYNAMIC MEMORY ALLOCATION (DMA) – <stdlib.h>

 **Aim:** To break the fixed-size limitations of standard arrays by allocating, resizing, and freeing heap memory at runtime using `malloc()`, `calloc()`, `realloc()`, and `free()`.

1. STATIC VS. DYNAMIC ALLOCATION (STACK VS. HEAP)

Static / Stack Memory:

- Fixed size decided at compile time (e.g., `int a[100];`).
- Cannot grow if more users enter; wastes space if empty.
- Automatically cleared when function ends.

Dynamic / Heap Memory:

- Exact size requested at runtime from OS using pointers.
- Persists until explicitly released using `free()`.
- Managed manually by programmer via `<stdlib.h>`.

2. THE 4 CORE DMA FUNCTIONS

1. `malloc()`

Memory Allocation: Allocates a single raw block of specified bytes.

```
ptr = (int*) malloc(n * sizeof(int));
```

Leaves memory with **garbage values**.

2. `calloc()`

Contiguous Allocation: Takes 2 arguments: number of items & size per item.

```
ptr = (int*) calloc(n, sizeof(int));
```

Initializes all allocated bytes to **Zero (0)**!

3. `realloc()`

Re-allocation: Expands or shrinks an already allocated heap block.

```
ptr = realloc(ptr, new_size);
```

Preserves original data during resizing.

4. `free()`

De-allocation: Returns the allocated heap memory back to the Operating System.

```
free(ptr); ptr = NULL;
```

Prevents dangerous **Memory Leaks**.

3. COMPLETE RUNTIME DEMONSTRATION (ALLOCATE, INPUT, RESIZE & FREE)

```
#include <stdio.h>
#include <stdlib.h> // Required for DMA

int main() {
    int n = 3, *ptr, sum = 0;

    // 1. Allocate heap space for 3 integers
    ptr = (int*) malloc(n * sizeof(int));

    // Always check if memory
```

Terminal Execution (Runtime):

Enter 3 elements:

10 20 30

Initial Sum = 60

Enter 1 extra element: **40**

Updated Total Sum = 100

Why Typecast (int*)?

Functions like `malloc()` return a generic pointer (`void*`) because they only allocate raw bytes without knowing what data type you want to store.

```

allocation succeeded
    if (ptr == NULL) {
        printf("Memory Allocation
Failed!\n");
        return 1;
    }

    printf("Enter %d elements:\n",
n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &ptr[i]);
        sum += ptr[i];
    }
    printf("Initial Sum = %d\n",
sum);

    // 2. Expand array to 4 elements
at runtime
    ptr = (int*) realloc(ptr, 4 *
sizeof(int));
    printf("Enter 1 extra element:
");
    scanf("%d", &ptr[3]);
    sum += ptr[3];
    printf("Updated Total Sum =
%d\n", sum);

    // 3. Free memory to prevent
memory leak
    free(ptr);
    ptr = NULL;
    return 0;
}

```

Prefixing with (int*) typecasts the memory chunk into an integer array pointer!

4. MALLOC() VS. CALLOC() COMPARISON

Feature	malloc()	calloc()
Syntax	void* malloc(size_t total_bytes);	void* calloc(size_t num_elements, size_t element_size);
Parameters	1 argument (total byte calculation)	2 arguments (number of items, size of each item)
Initial Values	Contains random garbage values	Initializes every single allocated byte to Zero (0)
Speed	Faster (does not spend CPU cycles clearing bytes)	Slightly slower (clears all bytes to 0)

⚠ Memory Leak & Dangling Pointer:

- **Memory Leak:** Forgetting to call free(ptr) consumes RAM permanently until program quits.
- **Dangling Pointer:** Accessing ptr after calling free(ptr) causes crashes! Always write ptr = NULL; immediately after freeing.

💡 Always Check for NULL:

If the computer runs out of RAM, malloc() returns NULL. Always verify if (ptr == NULL) before accessing array indices to prevent Segmentation Faults!



One-Line Summary: malloc() allocates uninitialized bytes, calloc() allocates zeroed memory, realloc() resizes existing heap blocks, and free() releases

memory back to the OS.

UNIT 3: C PROGRAMMING

PAGE 9: STRUCTURES (STRUCT) – USER-DEFINED DATA TYPES

 **Aim:** To group logically related variables of different data types under a single user-defined record using **struct**, access members using the dot (.) operator, read runtime input, and analyze physical memory allocation.

1. WHAT IS A STRUCTURE? & ARRAY VS. STRUCTURE

While an **Array** holds multiple items of the **same data type** (homogeneous), a **Structure** (defined by keyword **struct**) allows bundling multiple variables of **different data types** (heterogeneous) into one named blueprint.

Syntax: `struct TemplateName { member1; member2; ... }; → Always ends with a semicolon ;!`

 **Memory Layout for one variable `struct Student s1`; (Each member gets its own independent memory):**

101 s1.rollNo int (4 Bytes)	"Mukilan" s1.name char[20] (20 Bytes)	92.50 s1.marks float (4 Bytes)
--	--	---

⚡ Total Size $\approx$ `sizeof(rollNo) + sizeof(name) + sizeof(marks)` = **28 Bytes** (plus optional padding bytes for CPU alignment).

2. COMPLETE RUNTIME INPUT & DISPLAY PROGRAM

```
#include <stdio.h>

// 1. Blueprint declaration (no
memory allocated yet)
struct Student {
    int rollNo;
    char name[20];
    float marks;
}; // Semicolon is mandatory!

int main() {
    // 2. Variable creation (RAM
    allocated here)
    struct Student s1;

    printf("Enter Roll Number: ");
    scanf("%d", &s1.rollNo);

    printf("Enter Name: ");
    scanf("%s", s1.name); // Array
    name is address
```

Terminal Execution (Runtime):

```
Enter Roll Number: 101
Enter Name: Mukilan
Enter Marks: 92.5
```

--- Student Record ---

```
Roll No : 101
Name : Mukilan
Marks : 92.50
```

The Member Access Operator (.)

- Known as the **Dot Operator** or **Period Operator**.
- Syntax:
`variable_name.member_name`
- Directs the compiler to the relative memory offset of that exact field within the structure variable.

```

printf("Enter Marks: ");
scanf("%f", &s1.marks);

// 3. Printing details using Dot
(.) Operator
printf("\n--- Student Record ---
\n");
printf("Roll No : %d\n",
s1.rollNo);
printf("Name : %s\n", s1.name);
printf("Marks : %.2f\n",
s1.marks);

return 0;
}

```

3. ARRAY VS. STRUCTURE COMPARISON

Property	Array	Structure
Data Types	Homogeneous (all elements must be identical type)	Heterogeneous (members can be mixed: int, float, char)
Access Method	Accessed by numeric subscript index: <code>arr[0]</code>	Accessed by named identifier via dot operator: <code>s1.name</code>
Memory Allocation	Contiguous blocks of identical element size	Contiguous sequence of individual member types (+ padding)
Keywords	No special keyword needed (e.g., <code>int a[10];</code>)	Uses <code>struct</code> keyword (e.g., <code>struct Book b;</code>)

⚠ Semicolon Alert After Definition:

Forgetting the closing semicolon `};` after the structure definition is one of the most frequent beginner syntax errors in C! The compiler will flag errors on the lines that follow.

💡 Using typedef for Cleaner Syntax:

Typing `struct Student s1;` every time is lengthy. You can simplify it with `typedef`:
`typedef struct { int id; } Student;` → Now you can declare variables simply as `Student s1;`!



One-Line Summary: Structures bundle diverse data types into a single named record; individual members are accessed using the Dot (.) operator and stored sequentially in memory.

UNIT 3: C PROGRAMMING

PAGE 10: UNIONS (UNION) – SHARED MEMORY ALLOCATION

 **Aim:** To master user-defined types using the **union** keyword, understand the shared memory concept, observe value corruption when accessing multiple members simultaneously, and compare Structures vs. Unions.

1. WHAT IS A UNION? & SHARED MEMORY ARCHITECTURE

A **Union** is a user-defined data type similar to a structure, with one fundamental difference: **all members share the exact same memory space**. While a struct allocates memory for *every* member, a union's total size equals only the size of its **largest member**.

Syntax: `union TemplateName { member1; member2; ... };` → *Only one member can hold a meaningful value at any given time!*

struct Data (Independent Locations)

`i (4B)` `f (4B)` `ch (1B)`

Total = $4 + 4 + 1 = 9$ Bytes (Each has its own address)

union Data (One Shared Location)

[Shared 4 Bytes] ← `i` / `f` / `ch`

Total = $\max(4, 4, 1) = 4$ Bytes (All start at the exact same address!)

2. COMPLETE RUNTIME INPUT & OVERWRITE DEMONSTRATION

```
#include <stdio.h>

union Record {
    int id;
    float salary;
    char grade;
};

int main() {
    union Record r;

    // 1. Correct Usage: Use one member at a time
    printf("Enter ID: ");
    scanf("%d", &r.id);
    printf("Stored ID = %d\n", r.id);

    printf("Enter Salary: ");
    scanf("%f", &r.salary); // Overwrites id!
    printf("Stored Salary = %.2f\n", r.salary);
```

Terminal Execution (Runtime):

Enter ID: **101**
Stored ID = 101

Enter Salary: **45000.50**
Stored Salary = 45000.50

--- Inspecting Overwritten id ---
id value now: 1194270848
Size of union: **4 bytes**

Why Did ID Turn Into Garbage?

Because `id` and `salary` share the **exact same 4 bytes in RAM!** Writing `45000.50` stores IEEE-754 floating-point bits across those 4 bytes, wiping out the integer bits that represented `101`.

```

// 2. Observing Overwrite /
Corruption
printf("\n--- Inspecting
Overwritten id ---\n");
printf("id value now: %d
(Corrupted!)\n", r.id);
printf("Size of union: %lu
bytes\n", sizeof(r));

return 0;
}

```

3. DETAILED COMPARISON: STRUCTURE VS. UNION

Feature	Structure (struct)	Union (union)
Keyword	struct	union
Memory Allocation	Sum of all member sizes (+ padding bytes)	Equal to the size of the largest member only
Member Sharing	Each member has its own distinct address	All members share the single starting address
Value Retention	All members hold valid values simultaneously	Only one member holds a valid value at a time
Primary Use Case	Modeling real entities (Student, Employee, Point)	Hardware registers, memory conservation, variants

⚠ The Overwrite Trap:

Never write to member B and expect member A to retain its old data. Assigning a value to any member directly overwrites the bytes shared by all other members in that union!

💡 Real-World Use Case for Union:

Unions shine when designing a system where an item can be identified by *either* a Passport Number (string) *or* an Aadhaar Number (integer) — never both at once — saving precious RAM on embedded microcontrollers!



One-Line Summary: A Union allocates only enough memory for its largest member; all members share that single address space, so modifying one member overwrites the rest.

UNIT 3: C PROGRAMMING

PAGE 11: ARRAYS OF STRUCTURES – MANAGING MULTIPLE RECORDS

 **Aim:** To scale user-defined structures by declaring an array of structures, manage multiple real-world student/employee records using loops, read input at runtime, and visualize contiguous record storage in memory.

1. CONCEPT & MEMORY LAYOUT OF AN ARRAY OF STRUCTURES

A single structure variable stores details of only **one entity**. To store records for a whole class of 60 students or 100 employees, we create an **Array of Structures**.

Syntax: **struct StructureName array_name[size];** → e.g., `struct Student s[3];` reserves memory for 3 contiguous student records in RAM.

👁 **Contiguous Storage of struct Student s[3] (Each block = 1 complete record):**

s[0] (Record 1)
.id = 101
.name = "Arun"
.marks = 85.0
Address: #2000

s[1] (Record 2)
.id = 102
.name = "Bala"
.marks = 92.5
Address: #2028

s[2] (Record 3)
.id = 103
.name = "Charan"
.marks = 78.0
Address: #2056

⚡ Total Size = 3 × sizeof(struct Student). In RAM, s[1] begins immediately after s[0] finishes!

2. RUNTIME INPUT & DISPLAY PROGRAM (LOOP TRAVERSAL)

```
#include <stdio.h>

struct Student {
    int id;
    char name[20];
    float marks;
};

int main() {
    struct Student s[2]; // Array of
    2 records

    // 1. Reading input for multiple
    records
    for (int i = 0; i < 2; i++) {
        printf("Enter ID, Name,
        Marks for Student %d:\n", i + 1);
        scanf("%d %s %f", &s[i].id,
        s[i].name, &s[i].marks);
    }

    // 2. Displaying all student
    records
```

Terminal Execution (Runtime):

```
Enter ID, Name, Marks for Student 1:
101 Arun 85.0
Enter ID, Name, Marks for Student 2:
102 Bala 92.5
```

--- Student Database ---

```
[1] ID: 101 | Name: Arun | Marks: 85.00
[2] ID: 102 | Name: Bala | Marks: 92.50
```

Syntax Anatomy: **s[i].member**

- **s[i]**: Selects the whole structure instance at array index *i*.
- **.member**: Accesses the specific variable field inside that indexed structure.
- **Notice:** `&s[i].id` passes the memory address of the integer field inside the *i*-th record.

```

printf( \n--- Student Database
---\n");
for (int i = 0; i < 2; i++) {
    printf("[%d] ID: %d | Name:
%s | Marks: %.2f\n",
        i + 1, s[i].id,
        s[i].name, s[i].marks);
}
return 0;
}

```

3. COMMON ARRAY OF STRUCTURES OPERATIONS

Operation	Purpose	Code Logic Pattern
Linear Search	Find a record by specific ID or Name	if (s[i].id == searchId) { printf("Found: %s", s[i].name); }
Find Topper / Max	Scan all records to find highest mark	if (s[i].marks > max) { max = s[i].marks; topperIdx = i; }
Class Average	Sum a specific numeric member across all records	total += s[i].marks; → avg = total / n;
Sorting	Sort records based on a member field	Swap complete structure: struct Student temp = s[i]; s[i] = s[j]; s[j] = temp;

⚠ **Subscript vs Dot Precedence Trap:**
 Always write `s[i].id` and NOT `s.id[i]`!
 • `s[i].id`: The `id` field of the `i`-th student.
 • `s.id[i]`: Treats `id` itself as an array, which causes a syntax error!

💡 **Direct Assignment Feature:**
 In C, you can copy an entire structure record to another in one single line: `s[0] = s[1];`! C automatically copies every individual field (including internal string arrays) byte-by-byte.

📄 **One-Line Summary:** An Array of Structures stores multiple composite records side-by-side in RAM; access individual fields using `array_name[i].member` inside standard loops.

UNIT 3: C PROGRAMMING

PAGE 12: FILE OPERATIONS – OPEN, WRITE, READ & CLOSE

 **Aim:** To achieve permanent data storage by creating and connecting to files on the secondary storage drive (Hard Disk) using C's standard file stream pipeline: `fopen()`, `fprintf()`, `fscanf()`, and `fclose()`.

1. WHY FILES? & THE 4-STAGE FILE LIFECYCLE

Variables in RAM vanish the instant a program terminates (**Volatile Memory**). **Files** store data persistently on non-volatile disks. Communication between C code and a disk file operates through an OS buffer called a **Stream** managed via a `FILE*` handle.



2. COMPLETE RUNTIME PROGRAM (WRITE THEN READ FROM FILE)

```
#include <stdio.h>

int main() {
    FILE *fp;
    int roll, readRoll;
    char name[30], readName[30];

    // --- PART 1: WRITING DATA TO FILE ---
    fp = fopen("student.txt", "w");
    if (fp == NULL) { printf("Error opening file!\n"); return 1; }

    printf("Enter Roll No & Name to write: ");
    scanf("%d %s", &roll, name);
    fprintf(fp, "%d %s\n", roll, name); // Writes to disk
    fclose(fp); // Flushes buffer and saves
    printf("Data saved to student.txt successfully.\n\n");

    // --- PART 2: READING DATA FROM FILE ---
    fp = fopen("student.txt", "r");
    if (fp == NULL) { printf("File not found!\n"); return 1; }
```

 **Terminal Execution (Runtime):**
Enter Roll No & Name to write: **101**
Mukilan
Data saved to student.txt successfully.

--- Data Read from Disk File ---
Roll: 101 | Name: Mukilan

Why `fclose()` is Critical

- C holds file writes in a temporary RAM buffer for speed.
- **`fclose(fp)`** forces the OS to physically write (flush) any remaining data to the hard disk.
- Failing to close a file risks data loss and leaves OS file locks open!

```

    fscanf(fp, "%d %s", &readRoll,
readName); // Reads back
    printf("--- Data Read from Disk
File ---\n");
    printf("Roll: %d | Name: %s\n",
readRoll, readName);
    fclose(fp);
    return 0;
}

```

3. FUNDAMENTAL FILE ACCESS MODES IN C

Mode	Meaning / Action	What happens if file doesn't exist?	What happens to existing data?
"r"	Read only	Returns NULL (Cannot open)	Preserved (Read-only access)
"w"	Write only	Creates a new empty file	Erased! (Overwrites everything)
"a"	Append (add to end)	Creates a new empty file	Preserved (Appends new data to end)
"r+"	Read + Write	Returns NULL	Preserved (overwrites from start)

⚠ The "w" Mode Danger Trap:

Opening a file with mode "w" immediately truncates the file size to 0 bytes! Any previous contents are wiped instantly. If you want to keep old data and add new records, always choose mode "a" (append).

💡 Console vs File I/O Parallel:

Notice how simple the syntax bridge is:

- `printf( ... )` ↔ `fprintf(fp, ... )`
- `scanf( ... )` ↔ `fscanf(fp, ... )`

You only add the FILE *fp argument as the first parameter!



One-Line Summary: File handling persists data to disk: use `fopen()` with appropriate mode, write with `fprintf()` or read with `fscanf()`, and always close with `fclose()`.

UNIT 3: C PROGRAMMING

PAGE 13: TEXT FILES VS. BINARY FILES (FWRITE & FREAD)

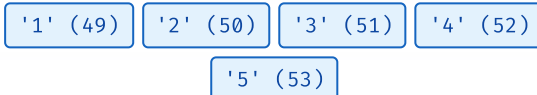
 **Aim:** To understand how text files differ from binary files in secondary storage, see how data types are converted or preserved, and perform raw block input/output using `fwrite()` and `fread()`.

1. HOW DATA IS STORED: TEXT VS. BINARY

In C, files are classified by how information is encoded onto the disk. **Text files** translate values into human-readable ASCII/UTF characters. **Binary files** dump the exact raw bytes from RAM directly to disk with zero conversion.

Text File (.txt) – Number 12345

Stored as 5 distinct character codes (1 byte each):



Requires 5 Bytes • Readable in Notepad

Binary File (.bin / .dat) – Number 12345

Stored directly as 4 raw memory bytes (int in RAM):



Requires 4 Bytes (fixed) • Opens as raw/gibberish symbols

2. COMPLETE BINARY FILE PROGRAM (FWRITE AND FREAD)

```
#include <stdio.h>

struct Player {
    int id;
    char name[20];
    int score;
};

int main() {
    FILE *fp;
    struct Player p1, p2;

    // 1. Reading runtime values to write
    printf("Enter ID, Name, Score: ");
    scanf("%d %s %d", &p1.id,
    p1.name, &p1.score);

    // 2. Writing raw struct block to binary file ("wb")
    fp = fopen("game.dat", "wb");
    fwrite(&p1, sizeof(struct Player), 1, fp);
    fclose(fp);
```

 **Terminal Execution (Runtime):**
Enter ID, Name, Score: **7 Dhoni 183**
Binary record written successfully.

--- Read from Binary File ---
ID: 7 | Name: Dhoni | Score: 183

Syntax of `fwrite()` & `fread()`

Both functions take 4 arguments:
`fwrite(ptr, size, count, fp);`

- **ptr:** Address of source/target memory buffer.
- **size:** Size in bytes of one record (sizeof).
- **count:** Number of records to transfer.
- **fp:** Target FILE* stream pointer.

```

printf("Binary record written\n\n");

// 3. Reading exact raw block
back("rb")
fp = fopen("game.dat", "rb");
fread(&p2, sizeof(struct
Player), 1, fp);
fclose(fp);

printf("--- Read from Binary
File ---\n");
printf("ID: %d | Name: %s |
Score: %d\n", p2.id, p2.name,
p2.score);
return 0;
}

```

3. KEY DIFFERENCES: TEXT FILE VS. BINARY FILE

Aspect	Text Files	Binary Files
Storage Format	Plain text (ASCII or UTF-8 characters)	Exact binary memory image (0s and 1s)
Human Readability	Easily opened & edited in text editors (Notepad)	Unreadable directly; opens as gibberish symbols
I/O Functions	fprintf(), fscanf(), fputs(), fgets()	fwrite(), fread()
Opening Modes	"r", "w", "a"	"rb", "wb", "ab" (notice the 'b'!)
Speed & Efficiency	Slower (requires number ↔ text conversions)	Much faster (direct RAM-to-disk block transfer)
End-of-Line Conversion	Converts \n to \r\n automatically on Windows	No newline transformation; raw bytes preserved

⚠ Forgetting the 'b' in File Modes:

Always include 'b' in mode strings ("wb", "rb") when working with binary files! On Windows, omitting 'b' makes the OS treat byte 0x1A as EOF and alters newline bytes, corrupting your data.

💡 Why Binary is Best for Structs:

Writing a structure to a text file requires formatting each field with fprintf. With binary files, one single line: fwrite(&s, sizeof(s), 1, fp); saves the whole record instantly in one CPU block!

 **One-Line Summary:** Text files store characters readable by humans; binary files store raw bytes using fwrite() and fread() with modes like "wb" and "rb" for faster, compact storage.

UNIT 3: C PROGRAMMING

PAGE 14: FILE POINTERS & RANDOM ACCESS (FSEEK, FTELL, REWIND)

 **Aim:** To master the internal file cursor/pointer, navigate freely across file bytes using random access, read specific data without sequentially scanning from the beginning, and track cursor positions using `fseek()`, `ftell()`, and `rewind()`.

1. WHAT IS A FILE POINTER & CURSOR POSITION?

A `FILE *fp` is a pointer to an internal C runtime control structure that tracks everything about an open file, including its buffer, mode, and crucially, its **current byte offset position (the cursor)**. Normally, reading or writing moves this cursor forward **sequentially**. But using C's random access functions, you can jump directly to any byte location!

 **Visualizing File Cursor in "data.txt" with text "ABCDEFGH":**

'A' Byte 0	'B' Byte 1	'C' Byte 2	'D' Byte 3 ← fp	'E' Byte 4	'F' Byte 5	'G' Byte 6	'H' Byte 7
---------------	---------------	---------------	--------------------	---------------	---------------	---------------	---------------

⚡ `fseek(fp, 3, SEEK_SET)` jumps straight to Byte 3 ('D') • `ftell(fp)` reports current offset: 3.

2. THE 3 RANDOM ACCESS NAVIGATION FUNCTIONS

1. `fseek()` – Move Cursor

Repositions file cursor to any byte offset.
`fseek(fp, offset, origin);`
Returns 0 on success.

2. `ftell()` – Current Position

Returns current byte index from file start.
`long pos = ftell(fp);`
Helpful to compute total file size!

3. `rewind()` – Reset to Start

Snaps file cursor straight back to Byte 0.
`rewind(fp);`
Equivalent to `fseek(fp, 0, SEEK_SET)`.

3. COMPLETE PROGRAM: RANDOM FILE JUMPS & FILE SIZE

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("letters.txt",
    "w+");
    char ch;

    // 1. Write initial text to file
    fputs("ABCDEFGH", fp);
```

 **Terminal Execution (Runtime):**
Position after seek: 3
Character read: D
Character after rewind: A
Total File Size: 8 bytes

The 3 Origin Constants for `fseek()`

- **SEEK_SET (0):** Offset measured from the **start** of the file.
- **SEEK_CUR (1):** Offset measured

```

// 2. Jump to byte offset 3 (4th
letter)
fseek(fp, 3, SEEK_SET);
printf("Position after seek:
%ld\n", ftell(fp));
ch = fgetc(fp);
printf("Character read: %c\n",
ch); // Reads 'D'

// 3. Reset back to start with
rewind()
rewind(fp);
ch = fgetc(fp);
printf("Character after rewind:
%c\n", ch); // 'A'

// 4. Calculate total file size
fseek(fp, 0, SEEK_END); // Move
to end
printf("Total File Size: %ld
bytes\n", ftell(fp));

fclose(fp);
return 0;
}

```

relative to **current cursor** location.

• **SEEK_END (2)**: Offset measured from the **very end** of the file.

4. QUICK NAVIGATION REFERENCE TABLE

Operation Task	Code Instruction	Cursor Shift Result
Rewind to Beginning	<code>rewind(fp);</code> or <code>fseek(fp, 0, SEEK_SET);</code>	Cursor placed at Byte 0 (first character)
Skip forward 5 bytes	<code>fseek(fp, 5, SEEK_CUR);</code>	Advances cursor forward by 5 positions
Step backward 3 bytes	<code>fseek(fp, -3, SEEK_CUR);</code>	Moves cursor back by 3 positions
Find total file size	<code>fseek(fp, 0, SEEK_END);</code> long size = <code>ftell(fp);</code>	Returns total byte count of the file

⚠ Negative Offset Trap:

Never pass a negative offset with `SEEK_SET`! Writing `fseek(fp, -5, SEEK_SET)` tries to access bytes before the beginning of the file, which produces undefined behavior and errors.

💡 Why Random Access Matters:

Imagine a database with 100,000 student records! Instead of looping through all 100,000 records, you can compute: `fseek(fp, id * sizeof(struct Student), SEEK_SET);` to fetch record #50,000 instantly!



One-Line Summary: File pointers track the active byte cursor; navigate anywhere using `fseek()`, read your exact position with `ftell()`, and jump back to the start with `rewind()`.

UNIT 3: C PROGRAMMING

PAGE 15: ERROR HANDLING IN FILE OPERATIONS

 **Aim:** To prevent unexpected program crashes by detecting, diagnosing, and gracefully handling file runtime errors such as missing files, permissions issues, disk-full faults, and distinguishing between true I/O errors and natural End-Of-File (EOF).

1. WHY ERROR HANDLING MATTERS IN FILE I/O?

Unlike variables in RAM which are always accessible, files live in secondary storage controlled by the Operating System. File operations can fail for many external reasons: a deleted file, bad permissions, or a disconnected USB drive. Every file stream maintains internal **status flags** to track what happens during read/write calls:

EOF Flag (feof)

Set when a read operation tries to read **past the final byte** of the file. Not an error — just marks the end!

Error Flag (ferror)

Set when a physical **hardware or stream failure** occurs (e.g., writing to a read-only file or corrupted disk sector).

2. THE 4 ESSENTIAL ERROR-CHECKING TOOLS

1. feof(fp)

Returns non-zero if the file pointer has reached the End of File.

2. ferror(fp)

Returns non-zero if an I/O error occurred on the stream.

3. perror(str)

Prints your custom message followed by OS error explanation.

4. clearerr(fp)

Resets both EOF and error flags back to 0 for retry.

3. COMPLETE PROGRAM: DETECTING MISSING FILES & READING SAFELY

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    char filename[30], ch;

    printf("Enter file name to open:
");
    scanf("%s", filename);

    // 1. Check for NULL pointer on fopen()
    fp = fopen(filename, "r");
    if (fp == NULL) {
```

Terminal Case 1 (Missing File):

```
Enter file name to open: test.txt
Error: No such file or directory
```

Terminal Case 2 (Successful Read):

```
Enter file name to open: sample.txt
File Contents:
Hello C Programmers!
Reached End of File successfully.
```

What Does perror() Do?

- When any system call fails, C sets a global error integer named **errno**.
- `perror("CustomMsg")` converts that code into plain English (e.g.

```

    perror("Error"); // Prints
system error description
    return 1;
}

// 2. Read characters and
monitor stream flags
printf("\nFile Contents:\n");
while ((ch = fgetc(fp)) != EOF)
{
    putchar(ch);
}

// 3. Diagnose why the loop
stopped
if (ferror(fp)) {
    printf("\nError: An I/O
error occurred while reading!\n");
} else if (feof(fp)) {
    printf("\nReached End of
File successfully.\n");
}

fclose(fp);
return 0;
}

```

"Permission denied" or "No such file or directory") and prints it automatically!

4. QUICK ERROR FUNCTION COMPARISON TABLE

Function Prototype	Header	When to Call It?	Return Value
int feof(FILE *fp);	<stdio.h>	After a read operation finishes	Non-zero if EOF reached, else 0
int ferror(FILE *fp);	<stdio.h>	After reading or writing to verify stream integrity	Non-zero if an error occurred, else 0
void perror(const char *s);	<stdio.h>	Immediately after fopen() = NULL	None (prints error string directly to stderr)
void clearerr(FILE *fp);	<stdio.h>	Before retrying an operation on the same stream	None (clears both EOF and error flags)

⚠ **The while(!feof(fp)) Trap (Common Exam Bug):**
 feof() only returns true **after** a read attempt fails by hitting past EOF. Using while(!feof(fp)) { fscanf(...); print(...); } will process the last line **twice!** Always test the return of the read function: while(fscanf(...) == 1).

💡 **Always Guard fopen() with NULL Check:**
 Never use fp without testing if (fp == NULL) first! Passing a NULL pointer to fgetc() or fprintf() results in an immediate **Segmentation Fault / crash!**

📄 **One-Line Summary:** Check **fp == NULL** after opening; use **perror()** for human-readable error messages, and verify **feof()** vs **ferror()** to distinguish EOF from disk failures.

UNIT 3: C PROGRAMMING

PAGE 16: STANDARD C LIBRARIES & BUILT-IN HEADERS

 **Aim:** To master the core ANSI/ISO C standard library headers (<math.h>, <ctype.h>, <stdlib.h>, <time.h>), understand pre-compiled routines, and avoid reinventing standard utility algorithms.

1. STANDARD LIBRARY ARCHITECTURE & CORE HEADERS

The **C Standard Library** is a standardized collection of header files (.h) containing macro definitions, type definitions, and pre-compiled function declarations that provide cross-platform portability.

<math.h>

Mathematical routines for floating-point calculations.
sqrt(), pow(), abs(), ceil(), floor()

<ctype.h>

Character testing and case conversion routines.
isalpha(), isdigit(), toupper(), tolower()

<time.h>

System time query, clock ticks, and date conversions.
time(), clock(), difftime(), ctime()

2. COMPLETE RUNTIME DEMONSTRATION PROGRAM

```
#include <stdio.h>
#include <math.h> // sqrt, pow
#include <ctype.h> // toupper, isdigit
#include <stdlib.h> // abs, rand

int main() {
    double num = 16.0;
    char ch = 'm';
    char digit = '7';

    // 1. Math Library Functions
    printf("sqrt(%.1f) = %.2f\n", num, sqrt(num));
    printf("pow(2, 3) = %.2f\n", pow(2, 3));

    // 2. Character Handling Functions
    printf("toupper('%c') = '%c'\n", ch, toupper(ch));
    if (isdigit(digit)) {
        printf("%c is a valid numeric digit\n", digit);
    }
}
```

Terminal Execution (Runtime):

```
sqrt(16.0) = 4.00
pow(2, 3) = 8.00
toupper('m') = 'M'
'7' is a valid numeric digit
abs(-45) = 45
```

Compilation Note for <math.h>

- In GCC / Linux environments, mathematical functions reside in an external shared math library.
- You must link it by appending the **-lm** flag during compilation:
`gcc program.c -o program -lm`

```
// 3. General Utilities
printf("abs(-45) = %d\n",
abs(-45));
return 0;
}
```

3. QUICK FUNCTION MATRIX OF STANDARD HEADERS

Header File	Key Functions	Typical Use Case
<code><stdio.h></code>	printf, scanf, fopen, fclose, fgetc	Console and file stream input/output.
<code><stdlib.h></code>	malloc, free, exit, atoi, rand, qsort	Memory management, conversions, and process control.
<code><string.h></code>	strlen, strcpy, strcat, strcmp, memset	String manipulation and raw byte block operations.
<code><math.h></code>	sqrt, pow, sin, cos, log, ceil, floor	Scientific, geometric, and floating-point computations.
<code><ctype.h></code>	isalpha, isdigit, isalnum, toupper, tolower	ASCII character classification and case shifting.
<code><time.h></code>	time, clock, difftime, strftime	Benchmarking runtime and reading real-time calendar dates.

⚠ Implicit Declaration Warnings:

Using functions like `sqrt()` or `malloc()` without including their respective header files causes compiler warnings or faulty return-type assumptions (defaulting to `int` instead of pointers/doubles)!

💡 `ctype.h` vs ASCII Math:

Instead of manual ASCII subtraction like `ch - 32` to make letters uppercase, always use `toupper(ch)`! It cleanly handles non-alphabetic symbols without risk of data corruption.



One-Line Summary: C Standard Libraries bundle pre-compiled, tested, and portable routines; include the appropriate header (`<math.h>`, `<ctype.h>`, `<stdlib.h>`) to avoid manual re-implementation.